DATABASE CODING & NAMING STANDARDS

Enterprise Data Governance Framework | v2.0

Classification: Internal — Confidential

Applies to: Snowflake | Databricks / Delta Live Tables | dbt | Great Expectations

Document Owner	Status	Review Cycle	Last Updated
Data Quality Lead	Active	Quarterly	2025

1. Purpose & Scope

This document defines the authoritative database coding standards, naming conventions, data quality rules, enforcement mechanisms, and governance operating model for all enterprise data platforms — including Snowflake and Databricks/Delta Live Tables.

This is not a style guide. It is the binding enterprise data governance framework that governs how data assets are created, validated, owned, monitored, and evolved across all environments. Compliance is mandatory for all data engineering, analytics engineering, and BI teams.

> **Scope**
> All structured data objects — tables, views, materialized views, schemas, databases, and Unity Catalog catalogs — across DEV, STAGING, and PROD environments in Snowflake and Databricks.

2. Business Value & Governance Objectives

This framework delivers measurable outcomes:

- Faster, more consistent development cycles through shared, automatically enforced standards
- Reduced data defects and rework through automated validation at ingestion and transformation
- Improved data reliability for BI dashboards and all downstream consumers
- Enforced accountability via a named ownership model for every data asset
- Audit readiness through lineage tracking, change logs, and data classification
- Alignment with ISO 8000, GDPR, and CCPA requirements

The primary KPI for framework success is the nightly Data Quality Score (DQS) — a composite 0–100 metric per dataset published to the enterprise data health dashboard.

3. Governance Operating Model

3.1 Roles & Responsibilities (RACI)

Every data asset must have named individuals in each role. Unowned assets are non-compliant by definition.

Role	Typical Profile	Responsibilities	Accountable For
Data Owner	Business Lead / VP	Approves schema changes; sets business rules; signs off on SLAs	Accuracy; business rule correctness
Data Steward	Analytics Eng / DQ Lead	Monitors DQ scores; manages rule catalog; escalates violations	Day-to-day quality; threshold enforcement
Data Custodian	Data / Platform Engineer	Implements pipelines; applies naming standards; maintains CI/CD checks	Technical implementation; naming compliance
Data Consumer	BI Analyst / Downstream	Reports quality issues; documents usage in catalog	Correct consumption; no misuse of raw layers

3.2 Dataset Tier Classification

Every dataset must be assigned a tier. Tier governs monitoring frequency, SLA stringency, and change management requirements.

Tier	Definition	Examples	Minimum Requirements
1	Mission-critical. Feeds customer-facing products or regulatory reporting.	`fact_order`, `dim_customer`, `financial_ledger`	Real-time anomaly detection; DQS ≥ 95; 1-hour freshness SLA; data contract required; full column-level lineage

2	Operational. Feeds internal BI dashboards and analytics.	`mart_sales_summary`, `rpt_churn`	Daily automated test suite; DQS ≥ 85; 4-hour freshness SLA; lineage documented
3	Experimental or archival. No downstream dependencies.	`sandbox_*`, `raw_archive_*`	On-demand validation; DQS ≥ 70; no freshness SLA; naming compliance required

3.3 Change Management Lifecycle

- Proposal: Engineer opens a Change Request ticket (Jira) with object name, change type (breaking/non-breaking), business justification, rollback plan, and affected downstream consumers.
- Peer Review: A second Data Custodian verifies naming compliance, data type alignment, and test coverage.
- Steward Approval: Data Steward validates DQ rule impact and lineage updates.
- Owner Sign-off (Tier 1 only): Data Owner approves changes affecting business rules or SLAs.
- Deployment: Merged via CI/CD pipeline. Schema change automatically logged to the audit table.
- Post-Deploy Validation: Automated test suite runs within 30 minutes. DQS verified within 2 hours.

Breaking vs. Non-Breaking Changes
Breaking: dropping a column, renaming a column, changing a data type. Requires Owner sign-off + minimum 30-day consumer notice (Tier 1) or 14-day notice (Tier 2).
Non-breaking: adding a nullable column, adding an index, adding a view. Requires Steward approval only.

3.4 Exception Process

- Engineer submits an Exception Request via the governance portal: standard being violated, business justification, duration, and compensating controls.
- Steward reviews and approves or rejects within 3 business days.
- Approved exceptions are logged in the exceptions registry with an expiry date (maximum 90 days).
- Expired exceptions without renewal trigger an automatic remediation ticket.

4. Naming Convention Standards

4.1 Universal Rules

- All database object names use lowercase snake_case exclusively
- Only alphanumeric characters and underscores — no spaces, hyphens, or special characters
- Names must be descriptive and aligned with business meaning; abbreviations permitted only from the approved abbreviations registry
- Maximum 64 characters (Snowflake and Databricks support 255; enterprise cap is 64 for readability)
- No SQL reserved words as object names (see Appendix A)
- Singular form for table names: customer not customers, order not orders

4.2 Database & Schema Naming

Object Type	Pattern	Examples
Snowflake Database	`<domain>_<env>`	`commerce_prod` | `finance_dev` | `analytics_staging`
Databricks Catalog	`<domain>_<env>`	`commerce_prod` | `analytics_dev`
Schema	`<layer>`	`raw` | `staging` | `intermediate` | `marts` | `reporting`
Team-scoped Schema	`<team>_<layer>`	`finance_marts` | `ops_staging`

Environment prefix (dev/qa)	`dev_` | `qa_`	`dev_commerce_prod (Snowflake clone); qa_fact_order`

4.3 Layered Table Architecture (Medallion)

All tables must be prefixed by their architectural layer. Unprefixed tables in non-sandbox schemas are a deployment blocker.

Prefix	Layer	Medallion	Rules
`raw_`	Raw / Landing	Bronze	Exact copy of source. No transforms. Append-only. Must include source_system_id and audit_insertion_timestamp.
`stg_`	Staging	Bronze→Silver	Cleaned and typed. One-to-one with source entity. Deduplication applied. Standard audit columns present.
`int_`	Intermediate	Silver	Business logic applied. Joins and enrichments. Not exposed directly to BI consumers.
`dim_`	Dimension	Gold	Conformed dimensions. SCD handling documented. Surrogate key (_sk suffix) required. is_current_flag required.
`fact_`	Fact	Gold	Immutable grain-level events. FK references dim_ surrogate keys. Partitioned by date column.
`mart_`	Data Mart	Gold	Pre-aggregated for specific BI use case. Owned by a specific business domain.
`rpt_`	Report Layer	Presentation	Final presentation layer for dashboards. Minimal transformation from mart_.
`sandbox_`	Sandbox	Experimental	Developer workspace only. No production pipeline may read from sandbox_ tables.

4.4 Column Naming Standards

Column Type	Required Suffix	Data Type	Examples
Primary Key	`_id`	VARCHAR(36) / NUMBER	`customer_id` | `order_id`
Surrogate Key	`_sk`	NUMBER (IDENTITY)	`customer_sk` | `product_sk`
Foreign Key	`_id (ref table name)`	Matches PK type	`customer_id in fact_order`
Date	`_date`	DATE	`order_date` | `birth_date`
Timestamp	`_timestamp`	TIMESTAMP_TZ	`created_timestamp` | `updated_timestamp`
Boolean	`_flag`	BOOLEAN	`is_active_flag` | `is_deleted_flag`
Amount / Currency	`_amount`	NUMBER(18,4)	`order_amount` | `refund_amount`
Percentage	`_pct`	NUMBER(5,4)	`discount_pct` | `allocation_pct`
Count	`_count`	NUMBER	`line_item_count` | `retry_count`
Code / Enum	`_code`	VARCHAR	`country_code` | `status_code`
Name / Label	`_name`	VARCHAR	`customer_name` | `product_name`
Description	`_description`	VARCHAR	`product_description` | `failure_description`

Boolean Columns: _flag only — _flg is prohibited
Use _flag exclusively. Boolean columns must use positive logic: is_active_flag = TRUE means the record is active.

> Negative logic is prohibited: is_not_deleted_flag is a naming violation and must be refactored.

4.5 Constraint & Index Naming

Type	Pattern	Example
Primary Key	`pk_<table>`	`pk_customer`
Foreign Key	`fk_<table>_<ref_table>`	`fk_order_customer`
Unique Constraint	`uq_<table>_<columns>`	`uq_customer_email`
Index	`idx_<table>_<columns>`	`idx_order_customer_id_order_date`
Check Constraint	`chk_<table>_<rule>`	`chk_allocation_pct_max`

4.6 View Naming

View Type	Prefix	Use Case	Layer
Standard View	`vw_`	Business logic over base tables	marts
Secure View (Snowflake)	`svw_`	PII masking; row-level security	reporting
Materialized View	`mvw_`	Pre-computed aggregations for performance	marts
Dynamic Table (Snowflake)	`dyn_`	Continuous auto-refresh transformation	intermediate

4.7 Mandatory Audit Columns

All stg_, int_, dim_, fact_, mart_, and rpt_ tables must include these columns. Absence is a deployment blocker.

Column Name	Data Type	Purpose
`audit_insertion_timestamp`	TIMESTAMP_TZ	When the record was inserted into this layer
`audit_updated_timestamp`	TIMESTAMP_TZ	When the record was last modified (NULL for insert-only tables)
`audit_batch_id`	VARCHAR	Pipeline run identifier for lineage tracing
`source_system_id`	VARCHAR	Originating system code (e.g., SALESFORCE, ERP_SAP)
`record_hash`	VARCHAR(64)	SHA-256 hash of business key columns for deduplication and CDC
`is_current_flag`	BOOLEAN	SCD Type 2 currency indicator — dim_ tables only

4.8 Good vs. Bad Naming Examples

Non-Compliant (Prohibited)	Compliant (Required)	Violation Reason
`Orders`	`fact_order`	No layer prefix; not singular; mixed case
`CustomerID`	`customer_id`	camelCase is prohibited
`is_not_deleted_flg`	`is_active_flag`	_flg prohibited; negative logic; wrong suffix
`dt`	`order_date`	Non-descriptive abbreviation
`customer data`	`stg_customer`	Space in name; no layer prefix

`SELECT * FROM orders`	`SELECT order_id, customer_id FROM fact_order`	SELECT * prohibited in production code

5. Data Type Alignment & Constraints

Column name suffixes are a contract. A suffix-to-type mismatch is a schema violation and a deployment blocker.

Suffix	Required Type	Platform Notes	Validation Rule
`_id`	VARCHAR(36) or NUMBER	VARCHAR for UUIDs; NUMBER for sequences	NOT NULL; referenced by FK if PK
`_sk`	NUMBER	IDENTITY / auto-increment	NOT NULL; platform-generated
`_date`	DATE	Never TIMESTAMP or VARCHAR	Valid date range enforced by CHECK constraint
`_timestamp`	TIMESTAMP_TZ	Always timezone-aware	NOT NULL for audit columns
`_flag`	BOOLEAN	BOOLEAN on both platforms	NOT NULL; DEFAULT value required
`_amount`	NUMBER(18,4)	4 decimal places for currency precision	CHECK (value >= 0) for non-negative amounts
`_pct`	NUMBER(5,4)	Stored as decimal: 0.15 = 15%	CHECK (value BETWEEN 0 AND 1)

6. SQL Coding Standards

6.1 Core Rules

- Never use SELECT * in production code — enumerate all required columns explicitly
- Always use explicit JOIN syntax (INNER JOIN, LEFT JOIN) — implicit comma joins are prohibited
- Use meaningful aliases aligned with the table name (o for fact_order, c for dim_customer). Never a, b, x.
- All CTEs must be named for their business purpose — cte1, cte2, tmp are prohibited names
- Partition all large fact tables by a date column; document the partition key in the model header comment
- Use QUALIFY instead of a subquery for window function filtering in Snowflake
- Avoid correlated subqueries in production pipelines — use window functions or joins

6.2 Compliant SQL Pattern

```
-- COMPLIANT: Production SQL standard with CTE, explicit joins, and typed aliases
WITH active_customer AS (
    -- Customers with at least one order in the last 90 days
    SELECT
        c.customer_id,
        c.customer_name,
        c.customer_segment_code,
        MAX(o.order_date)   AS last_order_date,
        COUNT(o.order_id)   AS order_count,
        SUM(o.order_amount) AS lifetime_order_amount
    FROM stg_customer     AS c
    INNER JOIN fact_order AS o ON c.customer_id = o.customer_id
    WHERE o.order_date >= DATEADD('day', -90, CURRENT_DATE())
    GROUP BY c.customer_id, c.customer_name, c.customer_segment_code
)
SELECT
    customer_id,
    customer_name,
    customer_segment_code,
```

```
    last_order_date,
    order_count,
    lifetime_order_amount
FROM active_customer
ORDER BY lifetime_order_amount DESC;
```

6.3 Prohibited Patterns

Prohibited	Compliant Alternative	Reason
`SELECT * FROM fact_order`	`SELECT order_id, customer_id, order_date FROM fact_order`	Schema drift breaks consumers silently
`FROM orders o, customers c WHERE o.cid = c.id`	`FROM fact_order o INNER JOIN dim_customer c ON o.customer_id = c.customer_id`	Implicit joins: ambiguous and non-optimizable
`WHERE o.dt > '2024-01-01'`	`WHERE o.order_date > '2024-01-01'::DATE`	Non-descriptive alias; untyped literal

7. Data Quality Framework

7.1 Data Quality Scoring Model (DQS)

Every Tier 1 and Tier 2 dataset receives a nightly composite Data Quality Score (0–100). Scores are published to the enterprise data health dashboard. Any dataset below its tier threshold generates an automated remediation ticket.

Dimension	Weight	Calculation	Tier 1 Threshold
Completeness	30%	% non-null values in mandatory fields	≥ 99.5%; critical key columns = 100%
Uniqueness	20%	% records with non-duplicate primary keys	100% — zero duplicate PKs tolerated
Validity	20%	% records passing all business rules	≥ 99% of records pass all defined rules
Timeliness	20%	% records within freshness SLA	≥ 99% within 1-hour SLA
Referential Integrity	10%	% FK values with valid parent records	100% — zero orphan records tolerated

DQS Thresholds by Tier
Tier 1: DQS ≥ 95. Score < 90 triggers CRITICAL alert and pipeline pause.
Tier 2: DQS ≥ 85. Score < 75 triggers WARNING alert and auto-creates remediation ticket.
Tier 3: DQS ≥ 70. Score < 60 triggers INFO notification only.

7.2 SLA Definitions

Tier	Freshness SLA	Availability SLA	Incident Response SLA	Volume Anomaly
1	≤ 1 hour lag	99.9% uptime	Ack: 15 min; Resolve: 4 hours	Alert if ±20% of 7-day avg
2	≤ 4 hours lag	99.5% uptime	Ack: 1 hour; Resolve: 24 hours	Alert if ±30% of 7-day avg
3	≤ 24 hours lag	Best effort	Ack: 1 business day	No automated alert

7.3 Data Health Rules

- Primary keys: 0% null tolerance; 0% duplicate tolerance. Violations are deployment blockers.
- Foreign keys: 100% referential integrity required. Orphan records trigger CRITICAL alert for Tier 1.
- Mandatory audit columns (Section 4.7): 0% null tolerance in NOT NULL columns.
- Volume anomalies: row count deviating beyond tier threshold from 7-day moving average triggers automated alerting.
- Schema drift: any unplanned column addition, removal, or type change triggers CRITICAL alert and pipeline pause regardless of tier.

8. Data Quality Rules Catalog

The following catalog defines active data quality rules. Critical rules halt the downstream pipeline on failure. New rules require Data Steward approval before activation.

Rule ID	Rule Name	Target Table	Validation Logic	Severity	Expected Result	Action on Failure
DQR-001	Primary Key Not Null	`fact_order`	order_id IS NOT NULL	**Critical**	COUNT(*) = 0	Halt pipeline; page on-call Steward
DQR-002	Primary Key Uniqueness	`fact_order`	COUNT(order_id) = COUNT(DISTINCT order_id)	**Critical**	Result = TRUE	Halt pipeline; open P1 ticket
DQR-003	FK Integrity: Order → Customer	`fact_order`	LEFT JOIN dim_customer; NULL customer_id count = 0	**Critical**	0 orphan records	Alert Steward; block downstream marts
DQR-004	Allocation Pct Sum ≤ 100%	`fact_resource_allocation`	SUM(allocation_pct) per resource_id ≤ 1.0	**High**	All groups ≤ 1.0	Alert Data Owner; quarantine affected rows
DQR-005	Customer Name Completeness	`dim_customer`	customer_name IS NULL count = 0	**High**	COUNT(*) = 0	Alert Steward; log to DQ incident table
DQR-006	Order Amount Non-Negative	`fact_order`	order_amount >= 0 for all records	**High**	COUNT(violations) = 0	Quarantine negative rows; alert Steward
DQR-007	Order Date Not Future	`fact_order`	order_date <= CURRENT_DATE()	**Medium**	COUNT(violations) = 0	Log warning; flag affected records
DQR-008	Boolean Not Null: is_active_flag	`dim_customer`	is_active_flag IS NOT NULL	**Medium**	COUNT(*) = 0	Log warning; apply DEFAULT value

DQR-009	Freshness SLA: fact_order	`fact_order`	MAX(audit_insertion_timestamp) >= NOW() - 1 hour	**Critical**	Result = TRUE	Alert Steward; page on-call engineer
DQR-010	Country Code in Reference List	`dim_customer`	country_code IN (SELECT country_code FROM ref_country)	**Medium**	COUNT(violations) = 0	Log warning; route to Steward review

New Rule Template

Field	Instructions
Rule ID	DQR-XXX — assigned sequentially by Data Steward
Rule Name	Short descriptive title, max 60 characters
Target Table	Fully qualified: <schema>.<table_name>
Validation Logic	SQL expression evaluating to 0 violations on success, or dbt test name
Severity	Critical / High / Medium / Low — see Section 9.1 for definitions
Expected Result	Value the validation must return to PASS (e.g., COUNT(*) = 0)
Action on Failure	Halt pipeline | Alert role | Quarantine rows | Log only
Owner	Named Data Steward responsible for this rule
Effective Date	Date rule became active in production
Review Frequency	Quarterly for Critical/High; biannually for Medium/Low

9. Monitoring, Alerting & Incident Response

9.1 Severity Levels

Severity	Pipeline Impact	Notification	Resolution SLA
Critical	Pipeline halted immediately	PagerDuty + Slack #data-incidents + email to Data Owner	Ack: 15 min; Resolve: 4 hours (Tier 1)
High	Pipeline continues; downstream quarantined	Slack #data-quality + Jira ticket auto-created	Ack: 1 hour; Resolve: 24 hours
Medium	Pipeline continues; rows flagged	Jira ticket auto-created; daily DQ digest	Resolve within current sprint (2 weeks)
Low	No pipeline impact	Logged to DQ incident table only	Resolve within 30 days

9.2 Canonical Validation Queries

These are the reference implementations. In production they run via enforcement tooling (Section 10), not manually.

Null check — completeness (DQR-001 pattern)

```
SELECT COUNT(*) AS null_pk_count FROM fact_order WHERE order_id IS NULL;
-- Expected: null_pk_count = 0
```

Duplicate PK check — uniqueness (DQR-002 pattern)

```
SELECT order_id, COUNT(*) AS dup_count
FROM fact_order
GROUP BY order_id HAVING COUNT(*) > 1;
-- Expected: 0 rows returned
```

Referential integrity — orphan FK detection (DQR-003 pattern)

```
SELECT o.order_id, o.customer_id, o.order_date
FROM fact_order AS o
LEFT JOIN dim_customer AS c ON o.customer_id = c.customer_id
WHERE c.customer_id IS NULL;
-- Expected: 0 rows returned
```

Business rule — allocation sum (DQR-004 pattern)

```
SELECT resource_id, ROUND(SUM(allocation_pct),4) AS total_allocation
FROM fact_resource_allocation
GROUP BY resource_id HAVING SUM(allocation_pct) > 1.0;
-- Expected: 0 rows returned
```

Freshness SLA check (DQR-009 pattern)

```
SELECT
    MAX(audit_insertion_timestamp)        AS last_loaded_at,
    DATEDIFF('minute',
        MAX(audit_insertion_timestamp),
        CURRENT_TIMESTAMP())              AS minutes_since_load,
    CASE WHEN DATEDIFF('minute', MAX(audit_insertion_timestamp),
              CURRENT_TIMESTAMP()) > 60
         THEN 'SLA_BREACH' ELSE 'WITHIN_SLA' END AS sla_status
FROM fact_order;
-- Expected: sla_status = 'WITHIN_SLA'
```

Volume anomaly check

```
WITH daily AS (
    SELECT order_date, COUNT(*) AS cnt
    FROM fact_order
    WHERE order_date >= DATEADD('day',-8,CURRENT_DATE())
    GROUP BY order_date
),
baseline AS (SELECT AVG(cnt) AS avg_7d FROM daily WHERE order_date < CURRENT_DATE())
SELECT d.order_date, d.cnt, b.avg_7d,
    ROUND(ABS(d.cnt - b.avg_7d)/NULLIF(b.avg_7d,0)*100,2) AS pct_deviation
FROM daily d CROSS JOIN baseline b
WHERE d.order_date = CURRENT_DATE()
  AND ABS(d.cnt - b.avg_7d)/NULLIF(b.avg_7d,0) > 0.20;
-- Expected: 0 rows (deviation within ±20% of 7-day average)
```

10. Enforcement Tooling & Automation

Standards without automated enforcement are suggestions. Every standard in this document maps to an enforcement mechanism. Manual-only enforcement is not acceptable for Tier 1 or Tier 2 assets.

10.1 dbt (Analytics Engineering Layer)

- All stg_, int_, dim_, fact_, and mart_ models must have a schema.yml with column-level tests.
- Mandatory tests per model: not_null on all NOT NULL columns; unique on all PKs; relationships on all FKs; accepted_values on all _code and _flag columns.
- dbt model descriptions are required for all Tier 1 and Tier 2 models before promotion to production. Undocumented models fail CI.

```
# dbt schema.yml — fact_order example
models:
  - name: fact_order
    description: 'Grain: one row per order. Source: stg_order. Tier: 1. Owner: Commerce
Steward'
    columns:
      - name: order_id
        tests: [not_null, unique]
      - name: customer_id
        tests:
          - not_null
          - relationships:
              to: ref('dim_customer')
              field: customer_id
      - name: audit_insertion_timestamp
        tests: [not_null]
```

10.2 Great Expectations (Observability Layer)

- GE suites are required for all raw_ ingestion points and all Tier 1 fact and dimension tables.
- Required expectations: expect_column_values_to_not_be_null, expect_column_values_to_be_unique (PKs), expect_table_row_count_to_be_between (volume bounds), expect_column_max_to_be_between (date sanity).
- A failed checkpoint on a Tier 1 table halts the pipeline and triggers a Critical alert.

```
# Great Expectations checkpoint config
validations:
  - batch_request:
      datasource_name: snowflake_prod
      data_asset_name: fact_order
    expectation_suite_name: fact_order.critical_suite
action_list:
  - name: send_slack_notification_on_failure
    action:
      class_name: SlackNotificationAction
      slack_webhook: ${SLACK_WEBHOOK_DATA_INCIDENTS}
      notify_on: failure
```

10.3 Snowflake-Native (Tasks & Alerts)

- Snowflake Tasks run scheduled DQ rule checks on tables not managed by dbt.
- Snowflake Alerts enforce freshness SLA monitoring and volume anomaly detection.
- Dynamic Tables (dyn_ prefix) must define TARGET_LAG aligned to the tier SLA.

```
-- Snowflake Alert: Freshness SLA breach on fact_order
CREATE OR REPLACE ALERT alert_fact_order_freshness
  WAREHOUSE = monitoring_wh  SCHEDULE = '10 MINUTES'
  IF (EXISTS (
    SELECT 1 FROM fact_order
    HAVING DATEDIFF('minute', MAX(audit_insertion_timestamp), CURRENT_TIMESTAMP()) > 60
  ))
  THEN CALL SYSTEM$SEND_EMAIL(
    'data-steward@company.com',
    'CRITICAL: fact_order freshness SLA breached',
    'fact_order has not refreshed within the 60-minute Tier 1 SLA.'
  );
```

10.4 Databricks / Delta Live Tables

- All DLT pipelines must define expectations using @dlt.expect_or_fail (Critical rules) or @dlt.expect_or_drop (High rules) decorators.
- Delta table properties must include a comment with owner, tier, and grain description.

```
# Databricks DLT: Expectation pattern for Tier 1 tables
import dlt

@dlt.table(comment='fact_order | Grain: one row per order | Tier 1 | Owner: Commerce Steward')
@dlt.expect_or_fail('order_id_not_null',       'order_id IS NOT NULL')
@dlt.expect_or_fail('order_amount_non_neg',    'order_amount >= 0')
@dlt.expect_or_drop('customer_id_not_null',    'customer_id IS NOT NULL')
def fact_order():
    return dlt.read('stg_order').select(
        'order_id','customer_id','order_date','order_amount','audit_insertion_timestamp'
    )
```

10.5 CI/CD Enforcement

- All PRs touching data models must pass: SQLFluff linting (naming standard validation), dbt compile + dbt test --select state:modified+, and a GE checkpoint dry-run for affected tables.
- SQLFluff naming violations are treated as build failures — PRs do not merge with unresolved violations.
- Any PR removing or renaming a column on a Tier 1/2 table requires a lineage impact report attached as a PR comment.

11. Data Lineage, Classification & Catalog

11.1 Lineage Requirements

- Every Tier 1 and Tier 2 table must have documented lineage: source system → raw → staging → intermediate → serving layer.
- Column-level lineage is required for all Tier 1 fact and dimension tables; maintained in dbt's DAG and published to the catalog on each production run.
- Breaking changes to a Tier 1/2 table require an impact analysis listing all downstream consumers before approval.

11.2 Data Classification

Classification	Definition	Platform Tag	Access Control
PII	Name, email, SSN, IP address, any personal identifier	`TAG: pii_sensitive`	Column masking policy (Snowflake) or Unity Catalog tag-based access (Databricks)
Financial	Revenue, cost, margin, payment card data	`TAG: financial_sensitive`	Role-based access; SOX compliance logging required
Confidential	Internal strategic data not for public disclosure	`TAG: confidential`	Restricted to named roles; no export to sandbox
Internal	Standard operational data, no special sensitivity	`TAG: internal`	Standard employee access
Public	Approved for external sharing or public dashboards	`TAG: public`	No restriction; must be explicitly approved by Data Owner

11.3 Data Catalog Requirements

- All Tier 1 and Tier 2 tables must be registered in the enterprise data catalog (dbt docs / Atlan / Collibra / Snowflake Information Schema as applicable).
- Required catalog fields per table: business description, data owner, data steward, tier, sensitivity classification, source system, refresh frequency, DQS score link, lineage link.
- Tables without catalog entries are flagged as non-compliant in the weekly governance report.

12. Data Contracts

Data contracts are binding agreements between data producers and data consumers. Mandatory for all Tier 1 assets; recommended for Tier 2.

Contract Element	Definition & Requirement
Schema Definition	Exact column names, data types, nullability — versioned in source control. Consumers build on declared schema only.
Semantics	Business meaning of each field in plain language. Agreed upon by Data Owner and consumer leads.
Freshness SLA	Maximum acceptable lag. Producer is responsible for resolution when breached.
Quality Guarantees	Which DQ rules are guaranteed by the producer. Consumers may not assume fields or quality levels outside the contract.
Change Policy	Non-breaking: 7-day notice. Breaking changes: 30-day notice (Tier 1); 14-day notice (Tier 2).
Violation Escalation	If the producer breaches a contract term, Data Steward escalates to Data Owner within 2 hours.

13. Governance, Compliance & Audit

13.1 Version Control & Change Log

- All schema definitions, dbt models, and DQ rule configurations must be version-controlled in Git. Direct production edits are prohibited.
- Every merged PR changing a schema object must include a CHANGELOG.md entry: change type, object, author, date, ticket reference.

13.2 Audit Table

All schema changes, DQ violations, and governance approvals are logged to the central immutable audit table.

```
CREATE TABLE governance.audit_log (
    audit_id             VARCHAR(36)   NOT NULL,   -- UUID
    event_timestamp      TIMESTAMP_TZ  NOT NULL,
    event_type           VARCHAR(50)   NOT NULL,   -- SCHEMA_CHANGE | DQ_VIOLATION | APPROVAL |
EXCEPTION
    object_schema        VARCHAR(128),
    object_name          VARCHAR(128),
    event_description    VARCHAR(2000),
    changed_by           VARCHAR(128)  NOT NULL,   -- email / service account
    ticket_reference     VARCHAR(64),
    dq_rule_id           VARCHAR(20),              -- DQR-XXX if applicable
    severity             VARCHAR(20),
    resolution_timestamp TIMESTAMP_TZ,
    CONSTRAINT pk_audit_log PRIMARY KEY (audit_id)
);
```

13.3 Regulatory Alignment

- ISO 8000 (Parts 1, 2, 8): Data quality measurement and improvement cycle — implemented via the DQS model and quarterly catalog reviews.
- GDPR Article 5: Data minimisation and accuracy enforced through PII tagging, completeness thresholds, and validity rules.
- CCPA: Consumer data tagged pii_sensitive is governed by Snowflake column masking or Databricks Unity Catalog row filters.
- SOX (if applicable): Financial data requires access logging and end-to-end auditability via lineage and the audit log.

13.4 Governance Review Cadence

Activity	Frequency	Owner	Output
DQS Dashboard Review	Daily (automated)	Data Steward	Anomaly tickets auto-created
DQ Rules Catalog Review	Quarterly	Steward + Data Owners	Updated catalog; retired/new rules
Tier Classification Review	Quarterly	Data Owner	Updated tier assignments
Exception Registry Audit	Monthly	Data Steward	Expired exceptions remediated
Framework Document Review	Quarterly	Data Quality Lead	Version bump; change log updated
Lineage Completeness Audit	Monthly	Data Custodian	Undocumented assets flagged

14. Expected Outcomes & Success Metrics

Outcome	Measurement	12-Month Target
Improved data reliability	Average DQS across Tier 1 assets	≥ 97 out of 100
Reduced defects	Critical DQ incidents per month	< 2 per month
Faster delivery cycles	Time to merge a compliant schema change	< 2 business days
Increased data trust	% BI consumers rating data trustworthy (quarterly survey)	≥ 85%
Full ownership coverage	% of Tier 1/2 assets with named owner	100%
Naming compliance	% of new PRs passing CI naming checks first time	≥ 95%